



Photo by [rc.xyz NFT gallery](#) on [Unsplash](#)

★ Member-only story

Manage Environment Variables with Pydantic

Learn how to use pydantic-settings to validate your secrets



Marcello Politi · [Follow](#)

4 min read · Feb 27, 2025

Listen

Share

More

Introduction

Developers work on applications that are supposed to be deployed on some server in order to allow anyone to use those. Typically in the machine where these apps live,

developers set up environment variables that allow the app to run. These variables can be API keys of external services, URL of your database and much more.

For local development though, it is really inconvenient to declare these variables on the machine because it is a slow and messy process. So I'd like to share in this short tutorial how to use Pydantic to handle environment variables in a secure way.

.env file

What you commonly do in a Python project is to store all your environment variables in a file named `.env`. This is a text file containing all the variables in a `key : value` format. You can use also the value of one of the variables to declare one of the other variables by leveraging the `{}` syntax.

The following is an example:

```
#.env file

OPENAI_API_KEY="sk-your private key"
OPENAI_MODEL_ID="gpt-4o-mini"

# Development settings
DOMAIN=example.org
ADMIN_EMAIL=admin@${DOMAIN}

WANDB_API_KEY="your-private-key"
WANDB_PROJECT="myproject"
WANDB_ENTITY="my-entity"

SERPAPI_KEY= "your-api-key"
PERPLEXITY_TOKEN = "your-api-token"
```

Be aware the `.env` file should remain private, so it is important that this file is mentioned in your `.gitignore` file, to be sure that you **never push it on GitHub**, otherwise, other developers could steal your keys and use the tools you've paid for.

env.example file

To ease the life of developers who will clone your repository, you could include an `env.example` file in your project. This is a file containing only the keys of what is supposed to go into the `.env` file. In this way, other people know what APIs, tokens, or secrets in general they need to set to make the scripts work.

```
#env.example

OPENAI_API_KEY=""
OPENAI_MODEL_ID=""

DOMAIN=""
ADMIN_EMAIL=""

WANDB_API_KEY=""
WANDB_PROJECT=""
WANDB_ENTITY=""

SERPAPI_KEY= ""
PERPLEXITY_TOKEN = ""
```

python-dotenv

python-dotenv is the library you use to load the variables declared into the .env file. To install this library:

```
pip install python-dotenv
```

Now you can use the `load_dotenv` to load the variables. Then get a reference to these variables with the `os` module.

```
import os
from dotenv import load_dotenv

load_dotenv()

OPENAI_API_KEY = os.getenv('OPENAI_API_KEY')
OPENAI_MODEL_ID = os.getenv('OPENAI_MODEL_ID')
```

This method will first look into your .env file to load the variables you've declared there. If this file doesn't exist, the variable will be taken from the host machine. This means that you can use the .env file for your local development but then when the code is deployed to a host environment like a virtual machine or Docker container

we are going to directly use the environment variables defined in the host environment.

Pydantic

Pydantic is one of the most used libraries in Python for data validation. It is also used for serializing and deserializing classes into JSON and back. It automatically generates JSON schema, reducing the need for manual schema management. It also provides built-in data validation, ensuring that the serialized data adheres to the expected format. Lastly, it easily integrates with popular web frameworks like FastAPI.

pydantic-settings is a Pydantic feature needed to load and validate settings or config classes from environment variables.

```
!pip install pydantic-settings
```

We are going to create a class named `Settings`. This class will inherit `BaseSettings`. This makes the default behaviours of determining the values of any fields to be read from the `.env` file. If no var is found in the `.env` file it will be used the default value if provided.

```
from pydantic_settings import BaseSettings, SettingsConfigDict

from pydantic import (
    AliasChoices,
    Field,
    RedisDsn,
)

class Settings(BaseSettings):
    auth_key: str = Field(validation_alias='my_auth_key')
    api_key: str = Field(alias='my_api_key')

    redis_dsn: RedisDsn = Field(
        'redis://user:pass@localhost:6379/1', #default value
        validation_alias=AliasChoices('service_redis_dsn', 'redis_url'),
    )

    model_config = SettingsConfigDict(env_prefix='my_prefix_')
```

In the `Settings` class above we have defined several fields. The `Field` class is used to provide extra information about an attribute.

In our case, we setup a `validation_alias`. So the variable name to look for in the `.env` file is overridden. In the case reported above, the environment variable `my_auth_key` will be read instead of `auth_key`.

You can also have multiple aliases to look for in the `.env` file that you can specify by leveraging `AliasChoices(choise1, choise2)`.

The last attribute `model_config`, contains all the variables regarding a particular topic (e.g connection to a db). And this variable will store all `.env` var that start with the prefix `env_prefix`.

Instantiate and use settings

The next step would be to actually instantiate and use these settings in your Python project.

```
from pydantic_settings import BaseSettings, SettingsConfigDict

from pydantic import (
    AliasChoices,
    Field,
    RedisDsn,
)

class Settings(BaseSettings):
    auth_key: str = Field(validation_alias='my_auth_key')
    api_key: str = Field(alias='my_api_key')

    redis_dsn: RedisDsn = Field(
        'redis://user:pass@localhost:6379/1', #default value
        validation_alias=AliasChoices('service_redis_dsn', 'redis_url'),
    )

    model_config = SettingsConfigDict(env_prefix='my_prefix_')

# create immediately a settings object
settings = Settings()
```

Now what use the settings in other parts of our codebase.

```
from Settings import settings

print(settings.auth_key)
```

You finally have an easy access to your settings, and Pydantic helps you validate that the secrets have the correct format. For more advanced validation tips refer to the Pydantic documentation: <https://docs.pydantic.dev/latest/>

Final Thoughts

Managing the configuration of a project is a boring but important part of software development. Secrets like API keys, db connections are what usually power your application. Naively you can hardcode these variables in your code and it will still work, but for obvious reasons, this could not be a good practice. In this article, I showed you an introduction on how to use pydantic settings to have a structured and safe way to handle your configurations.

Follow me on [Medium](#) if you like this article! 😁

 [Linkedin](#) |  [X \(Twitter\)](#) |  [Website](#)

[Python](#)[Machine Learning](#)[AI](#)[Data Science](#)[Programming](#)[Follow](#)

Written by Marcello Politi

2.5K Followers · 59 Following

Machine Learning Scientist | Ex-ESA.



Responses (1)



Sandesh Chand

What are your thoughts?



Dua Ai she/her/him/them

Mar 1



Managing environment variables with Pydantic is an efficient way to handle configuration settings in Python applications, especially when dealing with complex or sensitive configurations.



8

[Reply](#)

[Open in app](#) ↗

Medium



Search



More from Marcello Politi



In TDS Archive by Marcello Politi

MIOps— A gentle introduction to Mlflow Pipelines

Orchestrate your end-to-end machine learning lifecycle with MLflow

★ Mar 13, 2024 🖱 714 💬 4



 In TDS Archive by Marcello Politi

Design Patterns with Python for Machine Learning Engineers: Template Method

Learn how to use the Template design pattern to enhance your code

★ Dec 24, 2024 🖱 160





In TDS Archive by Marcello Politi

Ace your Machine Learning Interview—Part 7

Dive into Ensemble Learning with Hard Voting Classifiers using Python

Nov 22, 2022 🖱 110



In TDS Archive by Marcello Politi

Data Engineering—ORM and ODM with Python

Manipulate database data leveraging an object-oriented programming paradigm



Jan 2



124

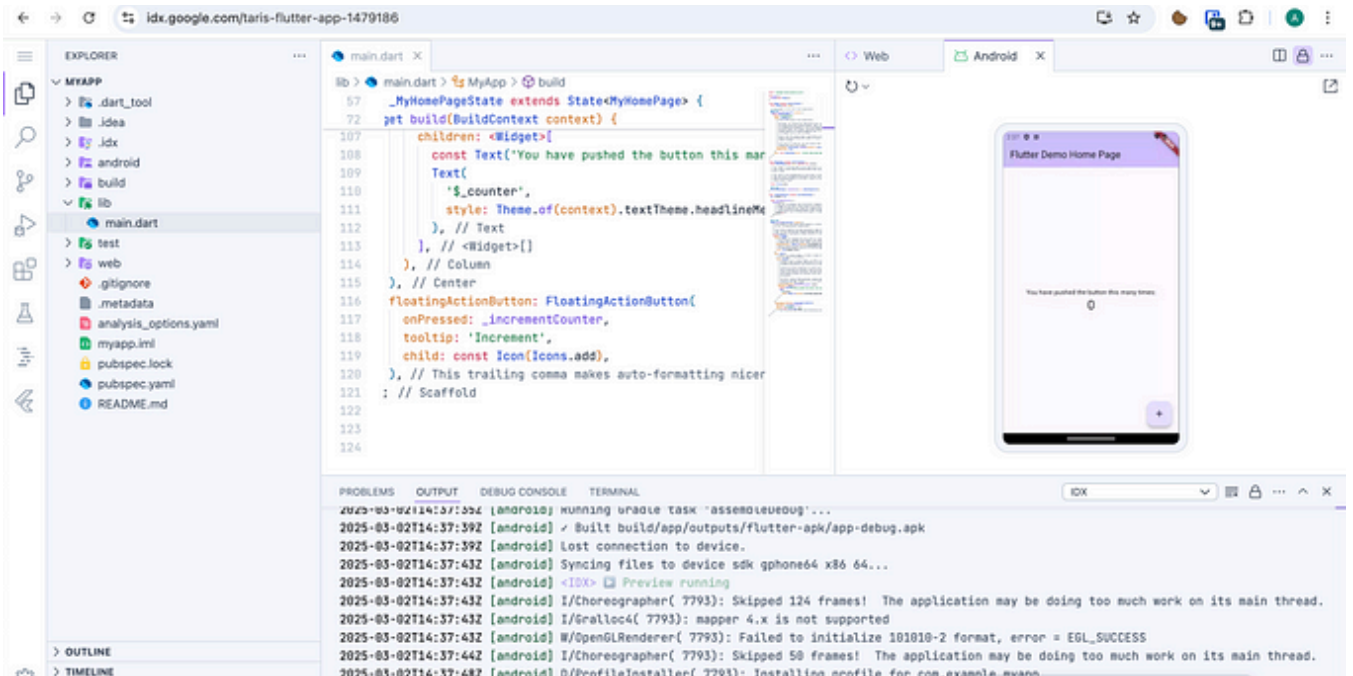


4



See all from Marcello Politi

Recommended from Medium



 In Coding Beauty by Tari Ibaba

This new IDE from Google is an absolute game changer

This new IDE from Google is seriously revolutionary.

★ Mar 11 🤝 3K 💬 183



 Sebastian Carlos

Fired From Meta After 1 Week: Here's All The Dirt I Got

This is not just another story of a disgruntled ex-employee. I'm not shying away from the serious corporate espionage or the ethical...

★ Jan 8 🖱 19K 💬 417



```
44 idfile = os.path.join(dataroot, "feature.bin")
45 self.names = [x.strip() for x in str.split(open(idfile).readlines())]
46 self.name2index = dict(zip(self.names, range(len(self.names))))
47 self.ndims = ndims
48 self.featurefile = os.path.join(dataroot, "feature.bin")
49 print "[BigFile] %d features, %d dimensions" % (len(self.names), self.ndims)
50 print "    binary: %s" % self.featurefile
51 print "    txt: %s" % idfile
52
53 def read(self, requested, isname=True):
54     if isname:
55         index_name_array = [(self.name2index[x], x) for x in requested if x in self.names]
56     else:
57         assert(min(requested) >= 0)
58         assert(max(requested) < len(self.names))
59         index_name_array = [(x, self.names[x]) for x in requested]
60         index_name_array.sort()
61     vecs = seq_read(self.featurefile, self.ndims, [x[0] for x in index_name_array])
62     return [x[1] for x in index_name_array], vecs
```



In Science Spectrum by Laurel W

Simple Ways to Tell if Python Code Was Written by an LLM

Yes, We Can Tell

★ Mar 22 🖱 801 💬 34



In Python in Plain English by Coders Stop

7 Python Libraries That Will Make Your Code Worse—But Your Life Better

Ever found yourself staring at elegant, production-ready code and thinking, “This is beautiful... but it took me three days to write”? You’re...

★ Mar 26 🖱 113 💬 1



11 OPEN-SOURCE PROJECTS THAT ARE LEGIT GAME-CHANGERS

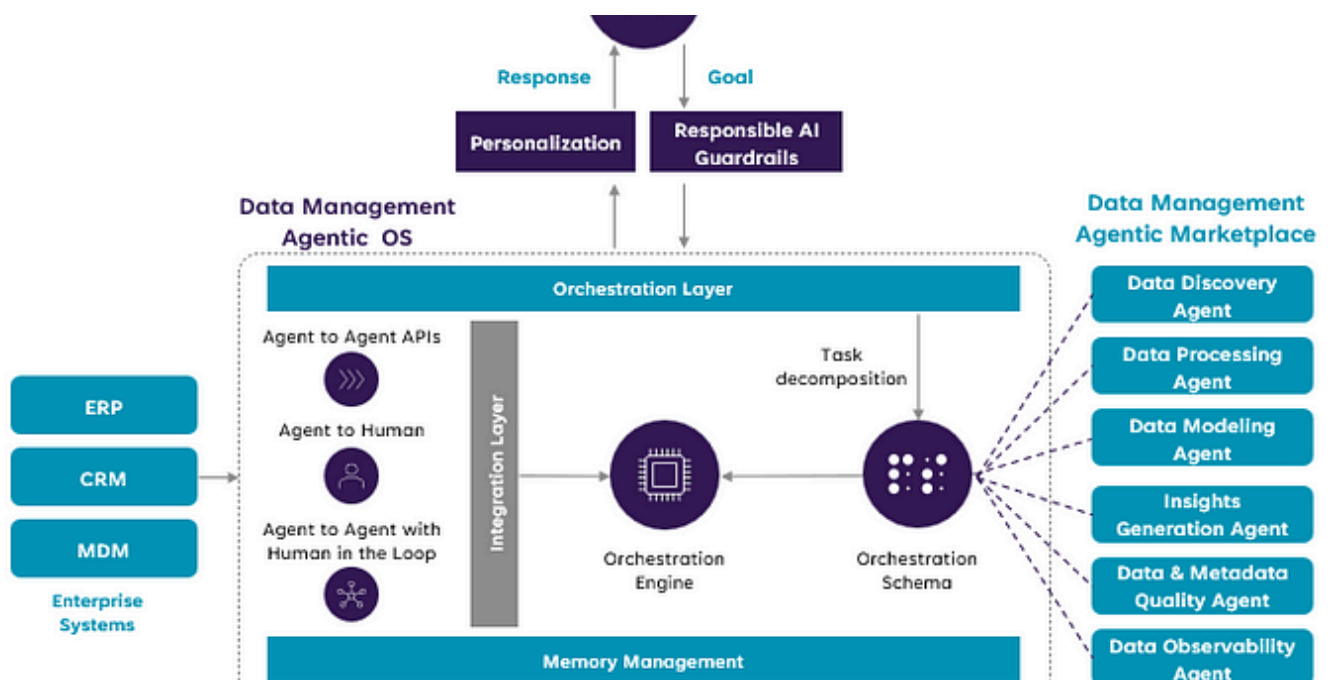


 In Let's Code Future by Let's Code Future

11 Open-Source Projects That are Legit Game-Changers

Really unique and valuable

★ Mar 24 🖱 996 💬 14



 In AI Advances by Debmalya Biswas

Agentic AI for Data Engineering

Reimagining Enterprise Data Management leveraging AI Agents

★ Mar 23 🖱️ 515 💬 12



See more recommendations